Sliding Window

> → **Key Idea**: Use a window that slides over array to track subset properties efficiently

> • **When to use**: Subarray problems, substring problems, fixed/variable window size optimization

> ⊕ **Complexity**: Time O(n), Space O(k) where k is window constraint

> ⚠ **Common Pitfall**: Not properly updating window state when sliding or expanding/contracting

Overview

Visual Example

Fixed Size Sliding Window

```
Find maximum sum subarray of size 3:
Array: [2, 1, 5, 1, 3, 2]

Window 1: [2, 1, 5] | 1, 3, 2    Sum = 8
Window 2:  2, [1, 5, 1] | 3, 2    Sum = 7
Window 3:  2, 1, [5, 1, 3] | 2    Sum = 9  ← Maximum
Window 4:  2, 1, 5, [1, 3, 2]     Sum = 6

🪟 Slide window one position at each step
```

Variable Size Sliding Window

```
Longest substring with at most 2 distinct characters:
String: "eceba"

Step 1: [e]           distinct=1 ≤ 2 ✓, expand
Step 2: [ec]          distinct=2 ≤ 2 ✓, expand
Step 3: [ece]         distinct=2 ≤ 2 ✓, expand
Step 4: [eceb]        distinct=3 > 2 ✗, shrink
Step 5:  e[ceb]       distinct=3 > 2 ✗, shrink
Step 6:  ec[eb]       distinct=2 ≤ 2 ✓, expand
Step 7:  ec[eba]      distinct=3 > 2 ✗, done

>> Best window: "ece" (length 3)
```

Timeline Visualization

```
Time:    0----1----2----3----4----5----6
Window:  [----][----][----][----]
Action:   add   add   add   rem   add   add
State:    1     2     3     2     3     4

>> Window expands/contracts based on constraints
```

The sliding window technique involves maintaining a "window" of elements and sliding it across the data structure to find subarrays/substrings that satisfy certain conditions.

This technique is particularly powerful for optimization problems on contiguous sequences, often reducing time complexity from $O(n^2)$ to $O(n)$.

Interview Tips

Pattern Recognition

- When to identify this technique in problem statements
- Key indicators and problem characteristics
- Similar problem types and variations

Common Mistakes

- Implementation pitfalls to avoid
- Edge cases that are often missed
- Performance considerations

Implementation Variations

```
// Alternative approaches and optimizations
// Template code for common variations
```

Time & Space Complexity

Pattern	Time Complexity	Space Complexity
Fixed Window	O(n)	O(1)
Variable Window	O(n)	O(k) where k is unique elements

Character Frequency	O(n)	O(1) or O(alphabet size)
Sliding Window Maximum	O(n)	O(k)

The sliding window technique typically achieves linear time complexity, making it highly efficient for subarray/substring optimization problems.

Common Problem Patterns

1. Maximum Sum Subarray of Size K

Problem: Find maximum sum of any contiguous subarray of size k.

```
public int maxSumSubarray(int[] nums, int k) {
    int windowSum = 0, maxSum = 0;
    for (int i = 0; i < nums.length; i++) {
        windowSum += nums[i];
        if (i >= k - 1) {
            maxSum = Math.max(maxSum, windowSum);
            windowSum -= nums[i - k + 1];
        }
    }
    return maxSum;
}
```

2. Longest Substring Without Repeating Characters

Problem: Find length of longest substring without repeating characters.

```
public int lengthOfLongestSubstring(String s) {
    Map<Character, Integer> map = new HashMap<>();
    int maxLen = 0, left = 0;

    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        if (map.containsKey(c)) {
            left = Math.max(left, map.get(c) + 1);
        }
        map.put(c, right);
        maxLen = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}
```

3. Minimum Window Substring

Problem: Find minimum window in s containing all characters of t.

```
public String minWindow(String s, String t) {
    Map<Character, Integer> need = new HashMap<>(), have = new HashMap<>();
    for (char c : t.toCharArray()) need.merge(c, 1, Integer::sum);

    int left = 0, minLen = Integer.MAX_VALUE, minStart = 0, formed = 0;

    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        have.merge(c, 1, Integer::sum);
        if (need.containsKey(c) && have.get(c).equals(need.get(c))) formed++;

        while (formed == need.size()) {
            if (right - left + 1 < minLen) {
                minLen = right - left + 1;
                minStart = left;
            }
            char leftChar = s.charAt(left);
            have.merge(leftChar, -1, Integer::sum);
            if (need.containsKey(leftChar) && have.get(leftChar) < need.get(leftChar
            left++;
        }
    }
    return minLen == Integer.MAX_VALUE ? "" : s.substring(minStart, minStart + minLe
}
```

4. Sliding Window Maximum

Problem: Return max sliding window of size k.

```
public int[] maxSlidingWindow(int[] nums, int k) {
    Deque<Integer> deque = new ArrayDeque<>();
    int[] result = new int[nums.length - k + 1];

    for (int i = 0; i < nums.length; i++) {
        while (!deque.isEmpty() && deque.peekFirst() < i - k + 1) deque.pollFirst();
        while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i]) deque.pollLast(
        deque.offerLast(i);
        if (i >= k - 1) result[i - k + 1] = nums[deque.peekFirst()];
    }
    return result;
}
```
